

Fazaia Bilquis College PAF Base NUR Khan, Rawalpindi



Mini Task Manager Project Operating System

Submitted by:

Taskeen Shoukat (12)

Javeria Kanwal (26)

Wajeeha Attiq (42)

Maheen Sajjad (77)

Submitted to:

Ms Tayyeba Naseer

Mini Task Manager Project

Introduction

The **Mini Task Manager** is a real-time process management tool designed to simulate core functionalities of an operating system's task manager. In this project, we focus on managing system processes through essential operations, including viewing a list of running tasks, suspending and resuming processes, killing processes, and safely exiting the system. The project is developed using **Dev**, and the primary goal is to create a functional and efficient tool for handling tasks in a real-time operating system environment.

In modern operating systems, **process management** plays a critical role in ensuring the smooth and efficient execution of applications. A **task manager** provides users with an interface to interact with these processes. This project mirrors key aspects of such an interface, but with a deeper understanding of process handling, which is essential for maintaining system stability and performance.

Functionalities of the Mini Task Manager:

1. List Of Running Process:

- The system provides a real-time display of all running processes. This includes essential information such as process ID (PID), process name. The list is continuously updated to reflect changes in the system's active processes.

2. Kill a Process:

- The tool allows users to terminate any process that may be unresponsive or consuming excessive resources. By selecting the process from the list, users can issue a command to immediately kill the process, freeing up system resources.

3. Suspend a Process:

- Users can temporarily suspend processes that are currently running. This is useful when a process needs to be paused for a short period without completely terminating it. Suspended processes can later be resumed without losing progress.

4. Resume a Process:

- After a process has been suspended, it can be resumed at any time. This operation restores the process to its original state, allowing it to continue executing as if it was never paused.

5. Exit:

- The exit functionality allows users to safely close the Mini Task Manager tool, ensuring that all processes are properly handled before the application shuts down. This ensures that no process is left in an unstable state.

Real-Time System:

This task manager operates in **real-time**, meaning that it continuously monitors the state of processes and updates the task list dynamically. Real-time systems require quick responses to any user input or changes in the process state, making it essential for the task manager to handle operations with minimal delay.

Code:

```
1  #include <windows.h>
2  #include <tlhelp32.h>
3  #include <iostream>
4  #include <iomanip>
5  #include <string>
6
7  using namespace std;
8
9  /* Utility Functions */
10
11 string getTimeStamp() {
12     SYSTEMTIME time;
13     GetLocalTime(&time);
14
15     char buffer[50];
16     sprintf(buffer, "%02d:%02d:%02d",
17             time.wHour, time.wMinute, time.wSecond);
18
19     return string(buffer);
20 }
21
22 void logMessage(string msg) {
23     cout << "[LOG " << getTimeStamp() << "]" << msg << endl;
24 }
25
26 // Simple progress effect
27 void showProgress() {
28     cout << "Processing";
29     for (int i = 0; i < 3; i++) {
30         Sleep(300);
31         cout << ".";
32     }
33     cout << endl;
34 }
35
```

Figure 1: C++ utility functions for a Mini Task Manager showing timestamp generation, logging with real-time output, and a simple progress indicator using Windows system APIs.

```

36 // Process Listing
37
38 void listProcesses() {
39     logMessage("Process list requested");
40
41     HANDLE snapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
42     PROCESSENTRY32 process;
43     process.dwSize = sizeof(PROCESSENTRY32);
44
45     if (snapshot == INVALID_HANDLE_VALUE) {
46         cout << "Error: Unable to create snapshot.\n";
47         return;
48     }
49
50     if (!Process32First(snapshot, &process)) {
51         cout << "Error: Unable to read process list.\n";
52         CloseHandle(snapshot);
53         return;
54     }
55
56     cout << "\n===== RUNNING PROCESSES =====\n";
57     cout << "PID\t\tProcess Name\n";
58     cout << "-----\n";
59
60     do {
61         cout << process.th32ProcessID << "\t\t"
62             << process.szExeFile << endl;
63     } while (Process32Next(snapshot, &process));
64
65     CloseHandle(snapshot);
66 }
67

```

Figure 2: C++ function for real-time process listing using Windows Tool Help APIs, displaying running processes with their Process IDs and executable names.

```

68 // Kill Process
69
70 void killProcess() {
71     DWORD pid;
72     cout << "\nEnter PID to terminate: ";
73     cin >> pid;
74
75     showProgress();
76
77     HANDLE hProcess = OpenProcess(PROCESS_TERMINATE, FALSE, pid);
78
79     if (hProcess == NULL) {
80         cout << "Error: Access denied or invalid PID.\n";
81         return;
82     }
83
84     if (TerminateProcess(hProcess, 0)) {
85         cout << "Process terminated successfully.\n";
86         logMessage("Process " + to_string(pid) + " terminated");
87     } else {
88         cout << "Failed to terminate process.\n";
89     }
90
91     CloseHandle(hProcess);
92 }
93

```

Figure 3: C++ implementation of the Kill Process functionality in a Mini Task Manager, allowing termination of a process by PID using Windows process management APIs with logging and progress feedback.

```

94 // Suspend Process
95
96 void suspendProcess() {
97     DWORD pid;
98     cout << "\nEnter PID to suspend: ";
99     cin >> pid;
100
101     showProgress();
102
103     HANDLE snapshot = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
104     THREADENTRY32 thread;
105     thread.dwSize = sizeof(THREADENTRY32);
106
107     bool found = false;
108
109     if (Thread32First(snapshot, &thread)) {
110         do {
111             if (thread.th32OwnerProcessID == pid) {
112                 HANDLE hThread = OpenThread(THREAD_SUSPEND_RESUME, FALSE,
113                                             thread.th32ThreadID);
114                 if (hThread) {
115                     SuspendThread(hThread);
116                     CloseHandle(hThread);
117                     found = true;
118                 }
119             }
120         } while (Thread32Next(snapshot, &thread));
121     }
122
123     CloseHandle(snapshot);
124
125     if (found) {
126         cout << "Process suspended successfully.\n";
127         logMessage("Process " + to_string(pid) + " suspended");
128     } else {
129         cout << "Error: Process not found or access denied.\n";
130     }
131 }
132

```

Figure 4: C++ function demonstrating how to suspend a Windows process by iterating through its threads using the Tool Help API (*CreateToolhelp32Snapshot*, *OpenThread*, *SuspendThread*).

```

133 // Resume Process
134
135 void resumeProcess() {
136     DWORD pid;
137     cout << "\nEnter PID to resume: ";
138     cin >> pid;
139
140     showProgress();
141
142     HANDLE snapshot = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
143     THREADENTRY32 thread;
144     thread.dwSize = sizeof(THREADENTRY32);
145
146     bool found = false;
147
148     if (Thread32First(snapshot, &thread)) {
149         do {
150             if (thread.th32OwnerProcessID == pid) {
151                 HANDLE hThread = OpenThread(THREAD_SUSPEND_RESUME, FALSE,
152                                             thread.th32ThreadID);
153                 if (hThread) {
154                     ResumeThread(hThread);
155                     CloseHandle(hThread);
156                     found = true;
157                 }
158             }
159         } while (Thread32Next(snapshot, &thread));
160     }
161
162     CloseHandle(snapshot);
163
164     if (found) {
165         cout << "Process resumed successfully.\n";
166         logMessage("Process " + to_string(pid) + " resumed");
167     } else {
168         cout << "Error: Process not found or access denied.\n";
169     }
170 }
171

```

Figure 5: C++ function illustrating how to resume a suspended Windows process by enumerating its threads and calling *ResumeThread* using the Tool Help API.

```

172 // CPU & Memory Usage
173
174 SIZE_T getProcessMemory(DWORD pid) {
175     SIZE_T memUsage = 0;
176     HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ, FALSE, pid);
177     if (hProcess) {
178         MEMORY_BASIC_INFORMATION mbi;
179         char* addr = 0;
180         while (VirtualQueryEx(hProcess, addr, &mbi, sizeof(mbi)) == sizeof(mbi)) {
181             if (mbi.State == MEM_COMMIT)
182                 memUsage += mbi.RegionSize;
183             addr += mbi.RegionSize;
184         }
185         CloseHandle(hProcess);
186     }
187     return memUsage;
188 }
189
190 void showProcessUsage() {
191     DWORD pid;
192     cout << "\nEnter PID to view CPU & Memory usage: ";
193     cin >> pid;
194
195     HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION, FALSE, pid);
196     if (hProcess == NULL) {
197         cout << "Error: Unable to open process.\n";
198         return;
199     }
200
201     FILETIME ftCreation, ftExit, ftKernel, ftUser;
202     if (GetProcessTimes(hProcess, &ftCreation, &ftExit, &ftKernel, &ftUser)) {
203         ULARGE_INTEGER kTime, uTime;
204         kTime.LowPart = ftKernel.dwLowDateTime;
205         kTime.HighPart = ftKernel.dwHighDateTime;
206         uTime.LowPart = ftUser.dwLowDateTime;
207         uTime.HighPart = ftUser.dwHighDateTime;
208
209         double cpuTime = (kTime.QuadPart + uTime.QuadPart) / 10000.0;
210         cout << fixed << setprecision(2);
211         cout << "CPU Time Used: " << cpuTime << " ms\n";
212     } else {
213         cout << "Unable to read CPU usage.\n";
214     }
215
216     SIZE_T mem = getProcessMemory(pid);
217     cout << "Memory Usage (approx.): " << mem / 1024 << " KB\n";
218
219     logMessage("Resource usage checked for PID " + to_string(pid));
220     CloseHandle(hProcess);
221 }

```

Figure 6: This C++ code utilizes the Windows API to query a specific process's memory footprint and cumulative CPU execution time based on its PID.

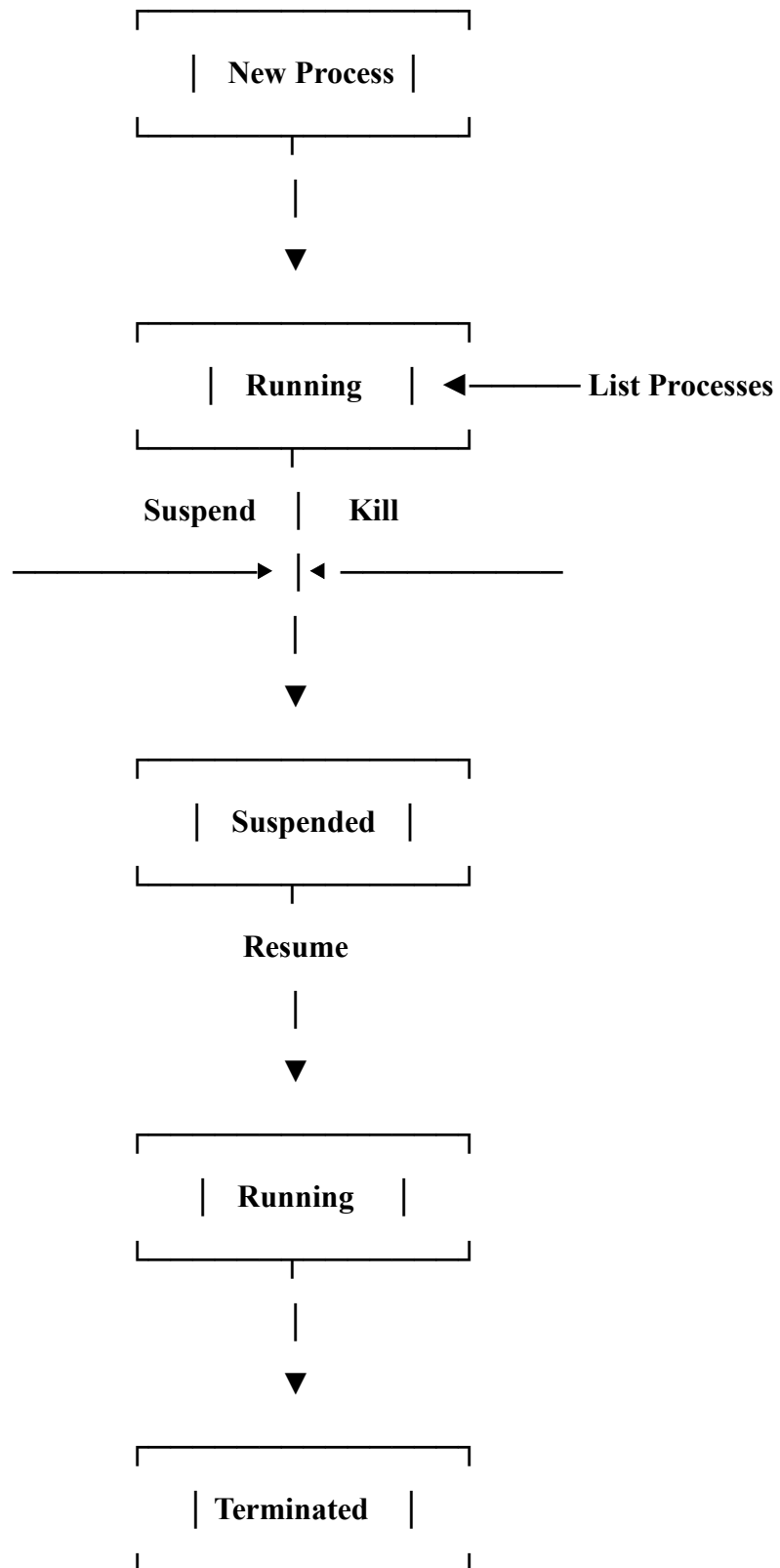
```

123 int main() {
124     int choice;
125
126     do {
127         SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), 12);
128         cout << "\n===== \n";
129         cout << " MINI TASK MANAGER (Process Manager)\n";
130         cout << " System Time: " << getTimeStamp() << endl;
131         cout << "===== \n";
132         cout << "1. List Of Running Processes\n";
133         cout << "2. Kill a Process\n";
134         cout << "3. Suspend a Process\n";
135         cout << "4. Resume a Process\n";
136         cout << "5. Show CPU & Memory Usage\n";
137         cout << "0. Exit\n";
138         cout << "Enter choice: ";
139         cin >> choice;
140
141         switch (choice) {
142             case 1:
143                 listProcesses();
144                 break;
145             case 2:
146                 killProcess();
147                 break;
148             case 3:
149                 suspendProcess();
150                 break;
151             case 4:
152                 resumeProcess();
153                 break;
154             case 5:
155                 showProcessUsage();
156                 break;
157             case 0:
158                 cout << "Exiting program...\n";
159                 break;
160             default:
161                 cout << "Invalid choice.\n";
162         }
163     } while (choice != 0);
164
165     return 0;
166 }

```

Figure 7: C++ code implements the main entry point for a terminal-based "Mini Task Manager" that uses a `do-while` loop and `switch` statement to handle process management commands like listing, killing, or monitoring resource usage.

Diagram:



Output:

```
=====
MINI TASK MANAGER (Process Manager)
System Time: 09:38:15
=====
1. List Running Processes
2. Kill a Process
3. Suspend a Process
4. Resume a Process
5. Show CPU & Memory Usage
0. Exit
Enter choice: |
```

Table 1: A command-line interface for a "Mini Task Manager" showing various process management options

```
Enter choice: 1
[LOG 09:39:02] Process list requested

===== RUNNING PROCESSES =====
PID      Process Name
-----
0        [System Process]
4        System
140      Secure System
184      Registry
604      smss.exe
1004     csrss.exe
808      wininit.exe
1128     services.exe
1136     lsass.exe
1156     lsass.exe
1324     svchost.exe
1360     fontdrvhost.exe
1384     WUDFHost.exe
1496     svchost.exe
1536     svchost.exe
1580     WUDFHost.exe
1800     svchost.exe
1852     svchost.exe
1864     svchost.exe
1888     svchost.exe
1908     svchost.exe
2036     svchost.exe
520      svchost.exe
8        svchost.exe
720      svchost.exe
2056     svchost.exe
2120     IntelCpHDCPSvc.exe
2136     svchost.exe
2148     svchost.exe
2232     svchost.exe
2244     svchost.exe
2260     svchost.exe
2440     svchost.exe
2556     svchost.exe
2716     svchost.exe
2724     svchost.exe
2804     svchost.exe
2896     svchost.exe
2624     svchost.exe
2492     svchost.exe
2796     svchost.exe
3100     svchost.exe
3116     svchost.exe
3280     Memory Compression
3312     svchost.exe
3336     svchost.exe
3384     WUDFHost.exe
3488     igfxCUIServiceN.exe
3532     svchost.exe
3540     svchost.exe
3548     svchost.exe
3712     svchost.exe
3816     WUDFHost.exe
3904     svchost.exe
3972     svchost.exe
4012     svchost.exe
4092     spoolsv.exe
3184     svchost.exe
4148     svchost.exe
4240     svchost.exe
4408     svchost.exe
4504     svchost.exe
4548     NgcIso.exe
4604     svchost.exe
4848     svchost.exe
4864     svchost.exe
4872     svchost.exe
4880     svchost.exe
```

Table 2: List of running process with PID

```
Enter choice: 3

Enter PID to suspend: 10276
Processing...
Process suspended successfully.
[LOG 09:41:17] Process 10276 suspended
```

Table 3: A command-line interface showing the successful suspension of process ID 10276 at 09:41:17.

```
Enter choice: 4

Enter PID to resume: 10276
Processing...
Process resumed successfully.
[LOG 09:57:09] Process 10276 resumed
```

Table 4: A command-line interface demonstrating the suspension, resumption, and resource usage tracking (CPU time and memory) for process ID 10276.

```
Enter choice: 5

Enter PID to view CPU & Memory usage: 10276
CPU Time Used: 1750.00 ms
Memory Usage (approx.): 439316 KB
[LOG 09:57:56] Resource usage checked for PID 10276
```

Table 5: A command-line interface tracking the complete life cycle of process ID 10276, including its suspension, resumption, resource usage monitoring, and final termination at 09:58:32.

```
Enter choice: 2

Enter PID to terminate: 10276
Processing...
Process terminated successfully.
[LOG 09:58:32] Process 10276 terminated
```

Table 6: A command-line interface tracking the complete life cycle of process ID 10276, including its suspension, resumption, resource usage monitoring, and final termination.

```
Enter choice: 0
Exiting program...

-----
Process exited after 1875 seconds with return value 0
Press any key to continue . . .
```

Table 7: Exit the system successfully.

Conclusion:

The Mini Task Manager successfully demonstrates the fundamental principles of process management in a real-time operating system environment. By providing functionality such as viewing running processes, suspending and resuming tasks, and terminating processes, the tool offers users direct control over system operations while maintaining stability and efficiency. Its real-time monitoring ensures that changes in process states are reflected instantly, enhancing system responsiveness. Overall, this project not only simulates the core features of a professional task manager but also provides practical insights into process handling, resource management, and the challenges of maintaining a responsive real-time system. It serves as both an educational tool and a foundation for further development in advanced system management applications.